

TanStack Query

Server State Made Simple

CSCI 39548 — Practical Web Development

Section Goals

- ▶ Replace `useEffect` + `fetch` with `useQuery`
- ▶ Write data with `useMutation` and invalidate the cache
- ▶ Understand the fresh/stale lifecycle and cache behavior
- ▶ Use React Query DevTools to inspect queries
- ▶ Gate queries with `enabled` for dependent fetches

Section Items

- ▶ useQuery
- ▶ Mutations
- ▶ DevTools and the Cache Lifecycle
- ▶ Dependent Queries and Polish

Remember React Snapshot 08?

- ▶ Three `useStates`: `data`, `loading`, `error`
- ▶ A `useEffect` with an `async` function, `try/catch/finally`
- ▶ And it **STILL** doesn't cache, dedupe, retry, or refetch
- ▶ That's the boilerplate for **one** fetch. Imagine ten.

Client State vs Server State

- ▶ `useState` / `localStorage` = **client state**. You own it. It's synchronous.
- ▶ Data on a server = **server state**. You don't own it. It's async, shared, can go stale, can fail.
- ▶ `useEffect` + `fetch` treats server state like client state — that's the mismatch
- ▶ **TanStack Query** is built for server state: caching, refetching, loading/error states, deduping — all handled

What We're Building

- ▶ Same Todo app. Same Tailwind styling.
- ▶ New: the data lives on a real server (`json-server`), and Query manages it

Section Items

► **useQuery**

- Mutations
- DevTools and the Cache Lifecycle
- Dependent Queries and Polish

The Setup: Provider

```
// main.tsx -- one client at the root
const queryClient = new QueryClient();

<QueryClientProvider client={queryClient}>
  <App />
</QueryClientProvider>
```

One QueryClient wraps the entire app. Every useQuery shares this cache.

The Setup: Query Function

```
// api.ts -- a plain async function. No React.  
export async function fetchTodos(): Promise<Todo[]> {  
  const res = await fetch("http://localhost:3001/todos");  
  if (!res.ok) throw new Error(`HTTP ${res.status}`);  
  return res.json();  
}
```

- ▶ Just a function that returns a Promise
- ▶ Throw on failure — Query catches it for you

The Setup: useQuery

```
const { data: todos, isLoading, isError, error } = useQuery({  
  queryKey: ["todos"],  
  queryFn: fetchTodos,  
});
```

- ▶ queryKey — the cache ID (an array)
- ▶ queryFn — how to fetch. Throw on failure.
- ▶ You get back data, isLoading, isError, error — for free
- ▶ The useEffect dance is gone

- ▶ Reload → exactly one GET /todos in the Network tab
- ▶ Toggle filters → no new request (it's cached)
- ▶ Compare line-for-line with React snapshot 08

Section Items

- useQuery

► **Mutations**

- DevTools and the Cache Lifecycle
- Dependent Queries and Polish

Reads vs Writes

- ▶ `useQuery` = read
- ▶ `useMutation` = write (POST / PATCH / DELETE)
- ▶ A mutation doesn't update the screen by itself
- ▶ So how does the list refresh?

The Invalidate Pattern

```
const queryClient = useQueryClient();

const addMutation = useMutation({
  mutationFn: addTodo,
  onSuccess: () =>
    queryClient.invalidateQueries({ queryKey: ["todos"] }),
});

addMutation.mutate({ text, minutes });
```

The Invalidate Chain

1. `mutate()` fires the server write
2. Server responds with success
3. `onSuccess` calls `invalidateQueries`
4. The `["todos"]` query is marked stale
5. `useQuery` refetches automatically
6. UI updates with fresh data

Why Invalidate Instead of Editing Local State?

- ▶ If you hand-edit a local array, the browser and server drift apart
- ▶ Invalidate-and-refetch keeps ONE source of truth (the server)
- ▶ Same one line works no matter how many components show that data

- ▶ Add a todo — watch TWO requests in the Network tab:
 - ▶ `POST /todos` (the write)
 - ▶ `GET /todos` (the automatic refetch)
- ▶ Add / toggle / delete, then **reload the page** — changes are still there

It Actually Persists

- ▶ Changes live on the server, not in component state
- ▶ Reload the page — everything is still there
- ▶ Open a second browser tab — same data

Pending State

```
<button disabled={isEmpty || addMutation.isPending}>  
  {addMutation.isPending ? "Adding..." : "Add"}  
</button>
```

- ▶ `isPending` is true while the server is processing
- ▶ Disable the button to prevent double-submits
- ▶ Throttle to Slow 3G in DevTools to actually see “Adding...”

Section Items

- useQuery
- Mutations
- ▶ **DevTools and the Cache Lifecycle**
 - Dependent Queries and Polish

React Query DevTools

```
import { ReactQueryDevtools } from
  "@tanstack/react-query-devtools";

// inside the provider:
<ReactQueryDevtools initialIsOpen={false} />
```

- ▶ Open the panel — every query, its state, its data, manual controls
- ▶ Stop guessing — see the cache

The Cache Lifecycle

```
fetch --> FRESH --(staleTime)--> STALE
      |                               |
      |                               |
served from cache,      refetched on a TRIGGER:
no network              remount, window focus,
                        reconnect, invalidate
```

staleTime Is a Gate, Not a Timer

```
new QueryClient({
  defaultOptions: {
    queries: { staleTime: 5000 },
  },
});
```

- ▶ `staleTime` = how long data is considered fresh
- ▶ Stale data does **NOT** refetch on its own
- ▶ It just becomes *eligible* to refetch on the next trigger
- ▶ Sit on the page doing nothing → no refetch, ever, even when stale

Want Polling? Different Knob.

```
useQuery({ ..., refetchInterval: 5000 }); // poll every 5s
```

- ▶ `refetchInterval` = the actual timer
- ▶ `staleTime` $\neq$ `refetchInterval` — don't conflate them

Refetch on Window Focus

- ▶ `refetchOnWindowFocus:` `true` (the default)
- ▶ Refetches **stale** queries when the tab regains focus
- ▶ Fresh query + refocus → nothing happens
- ▶ Stale query + refocus → instant refetch

- ▶ Open the React Query DevTools panel
- ▶ Tab away for $>5s$, come back $\rightarrow$ watch `["todos"]` refetch
- ▶ Tab away for $<5s$, come back $\rightarrow$ nothing happens (still fresh)

Section Items

- useQuery
- Mutations
- DevTools and the Cache Lifecycle
- ▶ **Dependent Queries and Polish**

One Query Waiting on Another

```
useQuery({  
  queryKey: ["user", assigneeId],  
  queryFn: () => fetchUser(assigneeId!),  
  enabled: assigneeId !== null,  
});
```

- ▶ `enabled: false` keeps a query from running
- ▶ Click a todo → `assigneeId` becomes a number → `enabled` flips → it fires
- ▶ The everyday use: “query B needs a value from user input or query A”

Dynamic Query Keys = Per-Item Cache

- ▶ `["user", 1]`, `["user", 2]` are separate cache entries
- ▶ Re-select a todo with an assignee you've seen → instant, no request

Live Code: Dependent Query

- ▶ Click todos with different assignees → watch keys accumulate in DevTools
- ▶ Click a repeat → cache hit, no network request

Error and Retry

```
const { isError, error, refetch, isFetching } = useQuery({
  ...
});

// in the error UI:
<button onClick={() => refetch()} disabled={isFetching}>
  {isFetching ? "Retrying..." : "Retry"}
</button>
```

Live Code: Error Recovery

- ▶ Kill the server → error state + Retry button
- ▶ Restart the server → click Retry → recovers, no page reload

What You Can Do Now

- ▶ Read server data with `useQuery` (no `useEffect`) ✓
- ▶ Write with `useMutation` + `invalidateQueries` ✓
- ▶ Read the cache in DevTools; reason about fresh/stale ✓
- ▶ Gate a query with `enabled` ✓

What We Skipped (Mention Only)

- ▶ **Optimistic updates** (`onMutate`) — update UI before the server replies
- ▶ **Pagination / infinite queries** (`useInfiniteQuery`)
- ▶ `select` to transform/slice cached data
- ▶ **Query key factories** for big apps

Before Next Class

- ▶ Swap one `useEffect` + `fetch` in your own code for `useQuery`
- ▶ Delete the loading/error `useStates` and feel how much disappears